

LAPORAN PRAKTIKUM

STRUKTUR DATA

Merge, Quick, dan Shell sort

disusun Oleh:

RIFKI MAULANA

NIM 2511533007

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM : ZAHARAN AZARIA ANVAYA



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 5 Juni 2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul “**Merge, Quick, dan Shell sort**”. Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai cara pemrograman struktur data Merge, Quick, dan Shell sort.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 5 Juni 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM.....	1
KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Kode Program	2
2.1.1 MergeSort.....	2
2.1.2 QuickSort	3
2.1.3 ShellSort	5
2.1.4 BubbleSortGUI	6
2.1.5 MergeSortGUI.....	11
BAB III KESIMPULAN	17
3.1 Kesimpulan.....	17
DAFTAR PUSTAKA.....	18
BAB IV Laporan Tugas	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Merge Sort adalah algoritma pengurutan yang bekerja dengan cara membagi data menjadi dua bagian secara berulang hingga setiap bagian hanya berisi satu elemen. Setelah itu, bagian-bagian tersebut digabungkan kembali dalam urutan yang benar. Quick Sort adalah salah satu algoritma pengurutan data yang paling cepat, yaitu dengan membagi list menggunakan sebuah pivot. Quick Sort juga menggunakan rekursif dalam algoritmanya. Shell Sort, juga dikenal sebagai metode Shell, adalah algoritma pengurutan perbandingan di tempat dan optimasi dari insertion sort . Algoritma ini meningkatkan efisiensi Insertion Sort dengan memungkinkan elemen dipindahkan dalam jarak yang lebih besar pada tahap awal, yang secara signifikan mengurangi jumlah pertukaran yang diperlukan, terutama untuk dataset yang lebih besar. Algoritma Algoritma ini berguna untuk menyorting suatu data dengan kecepatan dan dengan kebutuhan kekuatan komputasi tertentu.

1.2 Tujuan Praktikum

- Memahami konsep Sorting Merge, Quick, Shell dalam pemrograman.
- Mampu merancang dan membuat GUI dari sorting Merge dan Bubble pada java.

1.3 Manfaat Praktikum

- Menambah pemahaman mengenai penerapan Sorting dalam kasus kehidupan nyata.
- Melatih kemampuan membuat program yang terstruktur.
- Membantu memahami penggunaan class sebagai representasi objek di Java.

BAB II

PEMBAHASAN

2.1 Kode Program

2.1.1 MergeSort

```
package pekan8_2511533007;

public class MergeSort_2511533007 {
    void merge_3007(int arr_3007[], int l_3007, int m_3007, int r_3007) {
        // Find sizes of two subarrays to be merged
        int n1_3007 = m_3007 - l_3007 + 1;
        int n2_3007 = r_3007 - m_3007;
        /* Create temp arrays */
        int L[] = new int[n1_3007];
        int R[] = new int[n2_3007];
        /* Copy data to temp arrays */
        for (int i_3007 = 0; i_3007 < n1_3007; ++i_3007)
            L[i_3007] = arr_3007[l_3007 + i_3007];
        for (int j_3007 = 0; j_3007 < n2_3007; ++j_3007)
            R[j_3007] = arr_3007[m_3007 + 1 + j_3007];
        int i_3007 = 0, j_3007 = 0;
        // Initial index of merged subarrays array
        int k_3007 = l_3007;
        while (i_3007 < n1_3007 && j_3007 < n2_3007) {
            if (L[i_3007] <= R[j_3007]) {
                arr_3007[k_3007] = L[i_3007];
                i_3007++;
            } else {
                arr_3007[k_3007] = R[j_3007];
                j_3007++;
            }
            k_3007++;
        }
        /* Copy remaining elements of L[] if any */
        while (i_3007 < n1_3007) {
            arr_3007[k_3007] = L[i_3007];
            i_3007++;
            k_3007++;
        }
        /* Copy remaining elements of R[] if any */
        while (j_3007 < n2_3007) {
            arr_3007[k_3007] = R[j_3007];
            j_3007++;
            k_3007++;
        }
    }

    void sort_3007(int arr_3007[], int l_3007, int r_3007) {
        if (l_3007 < r_3007) {
            // Find the middle point
            int m_3007 = (l_3007 + r_3007) / 2;
            // Sort first and second halves
            sort_3007(arr_3007, l_3007, m_3007);
            sort_3007(arr_3007, m_3007 + 1, r_3007);
            // merge the sorted halves
            merge_3007(arr_3007, l_3007, m_3007, r_3007);
        }
    }

    /* A utility function to print array of size n */
    static void printArray_3007 (int arr_3007[]) {
        int n_3007 = arr_3007.length;
        for (int i_3007 = 0; i_3007 < n_3007; ++i_3007)
            System.out.print(arr_3007[i_3007] + " ");
        System.out.println();
    }

    public static void main (String args []) {
        int arr_3007[] = { 12, 11, 13, 5, 6, 7 };
        System.out.println("Sebelum terurut");
        printArray_3007(arr_3007);
        MergeSort_2511533007 ob_3007 = new MergeSort_2511533007();
    }
}
```

```

        ob_3007.sort_3007(arr_3007, 0, arr_3007.length - 1);
        System.out.println("\nSesudah Terurut menggunakan merge Sort");
        printArray_3007(arr_3007);
    }
}

```

Kode Program 2.1

Class ini berfungsi untuk mengimplementasikan algoritma Merge Sort dengan cara mengurutkan array angka dari nilai terkecil ke terbesar berbasis pendekatan divide and conquer. Di dalam class ini, method merge_3007 memproses penggabungan dua buah subarray yang telah terurut secara terpisah dengan cara membandingkan elemen-elemennya satu per satu, lalu menyalin sisa elemen yang belum terproses kembali ke array utama. Method sort_3007 berfungsi membagi array menjadi dua bagian secara rekursif hingga berukuran satu elemen sebelum akhirnya memanggil fungsi penggabungan. Sementara itu, method printArray_3007 merupakan fungsi pembantu untuk menampilkan seluruh elemen array ke layar, dan method main bertindak sebagai titik masuk program yang mendeklarasikan array angka acak, memanggil fungsi pengurutan, serta mencetak kembali hasil akhir array yang telah rapi.

```

Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13

```

2.1.2 QuickSort

```

package pekan8_2511533007;

public class QuickSort_2511533007 {
    static void swap_3007 (int[] arr_3007, int i_3007, int j_3007) {
        int temp_3007 = arr_3007[i_3007];
        arr_3007[i_3007] = arr_3007[j_3007];
        arr_3007[j_3007] = temp_3007;
    }
    // Metode tambahan untuk mengatur pivot menggubakan Median of three
    static void medianOfThree_3007 (int[] arr_3007, int low_3007, int high_3007) {
        int mid_3007 = low_3007 + (high_3007 - low_3007) / 2;

        // Urutkan elemen low, mid, dan high
        if (arr_3007[low_3007] > arr_3007[mid_3007]) {
            swap_3007(arr_3007, low_3007, mid_3007);
        }
        if (arr_3007[low_3007] > arr_3007[high_3007]) {
            swap_3007(arr_3007, low_3007, high_3007);
        }
        if (arr_3007[mid_3007] > arr_3007[high_3007]) {
            swap_3007(arr_3007, mid_3007, high_3007);
        }
        swap_3007 (arr_3007, mid_3007, high_3007);
    }
    static int partition_3007 (int[] arr_3007, int low_3007, int high_3007) {
        // Panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_3007(arr_3007, low_3007, high_3007);
    }
}

```

```

        int pivot_3007 = arr_3007[high_3007]; // Sekarang arr[high] sudah berisi nilai median
        int i_3007 = (low_3007 - 1);

        for (int j_3007 = low_3007; j_3007 <= high_3007 - 1; j_3007++) {
            // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
            if (arr_3007[j_3007] < pivot_3007) {
                // Increment indeks elemen yang lebih kecil
                i_3007++;
                swap_3007(arr_3007, i_3007, j_3007);
            }
        }
        swap_3007 (arr_3007, i_3007 + 1, high_3007);
        return (i_3007 + 1);
    }

    static void quickSort_3007(int[] arr_3007, int low_3007, int high_3007) {
        if (low_3007 < high_3007) {
            int pi_3007 = partition_3007 (arr_3007, low_3007, high_3007);
            quickSort_3007(arr_3007, low_3007, pi_3007 - 1);
            quickSort_3007(arr_3007, pi_3007 + 1, high_3007);
        }
    }

    public static void printArr_3007(int[] arr_3007) {
        for (int i_3007 = 0; i_3007 < arr_3007.length; i_3007++) {
            System.out.print(arr_3007[i_3007] + " ");
        }
        System.out.println();
    }

    public static void main (String[] args) {
        int[] arr_3007 = { 10, 7, 8, 9, 1, 5 };
        int N_3007 = arr_3007.length;
        System.out.print("Data sebelum diurutkan: ");
        printArr_3007(arr_3007);

        quickSort_3007(arr_3007, 0, N_3007 - 1);

        System.out.print("Data Terurut quicksort: ");
        printArr_3007(arr_3007);
    }
}

```

Kode Program 2.2

Class ini berfungsi untuk mengimplementasikan algoritma Quick Sort dengan bantuan strategi optimasi Median of Three untuk mengurutkan array dari terkecil ke terbesar. Di dalam class ini, method swap_3007 memproses pertukaran posisi antara dua elemen di dalam array berdasarkan indeksnya. Method medianOfThree_3007 berfungsi memilih pivot yang ideal dengan cara mengurutkan elemen awal, tengah, dan akhir dari subarray, lalu memindahkan nilai mediannya ke posisi paling kanan (sebagai pivot). Method partition_3007 memproses penataan ulang elemen sehingga angka yang lebih kecil dari pivot berada di sebelah kiri dan yang lebih besar berada di sebelah kanan. Method quickSort_3007 bertugas membagi dan menyelesaikan pengurutan subarray kiri dan kanan secara rekursif, dibantu oleh method printArr_3007 untuk mencetak elemen array ke layar. Sementara itu, method main bertindak

sebagai titik masuk program yang mendeklarasikan data acak awal, menjalankan proses pencarian posisi urut, dan menyajikan hasil akhirnya ke layar.

```
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

2.1.3 ShellSort

```
package pekan8_2511533007;

public class ShellSort_2511533007 {
    public static void shellSort_3007 (int[] A_3007) {
        int n_3007 = A_3007.length;
        int gap_3007 = n_3007 / 2;
        while (gap_3007 > 0) {
            for (int i_3007 = gap_3007; i_3007 < n_3007; i_3007++) {
                int temp_3007 = A_3007[i_3007];
                int j_3007 = i_3007;
                while (j_3007 >= gap_3007 && A_3007[j_3007 - gap_3007] > temp_3007) {
                    A_3007[j_3007] = A_3007[j_3007-gap_3007];
                    j_3007 = j_3007 - gap_3007;
                }
                A_3007[j_3007] = temp_3007;
            }
            gap_3007 = gap_3007 / 2;
        }
    }

    public static void main(String[] args) {
        int[] data_3007 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

        System.out.print("Sebelum: ");
        printArray_3007(data_3007);

        shellSort_3007(data_3007);

        System.out.print("Sesudah (Shell Sort): ");
        printArray_3007(data_3007);
    }

    public static void printArray_3007 (int[] arr_3007) {
        for (int i_3007 : arr_3007) System.out.print(i_3007 + " ");
        System.out.println();
    }
}
```

Kode Program 2.3

Class ini berfungsi untuk mengimplementasikan algoritma Shell Sort yang merupakan modifikasi optimasi dari Insertion Sort dengan memanfaatkan mekanisme jarak lompatan (gap). Di dalam class ini, method shellSort_3007 memproses pengurutan dengan membagi array ke dalam beberapa sub-list berdasarkan nilai gap tertentu yang terus mengecil setengahnya di setiap iterasi, lalu melakukan perbandingan dan pergeseran elemen yang serupa dengan Insertion Sort hingga gap bernilai nol. Method printArray_3007 berfungsi menampilkan seluruh elemen array ke baris teks layar menggunakan struktur perulangan for-each. Sementara itu, method main bertindak sebagai titik masuk program yang menginisialisasi

array berisi sepuluh angka acak, menampilkan kondisi awal, memicu proses pengurutan Shell Sort, dan memaparkan hasil akhir data yang telah sukses diurutkan.

```
Sebelum: 3 10 4 6 8 9 7 2 1 5  
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

2.1.4 BubbleSortGUI

```
1 package pekan8_2511533007;  
2  
3 import java.awt.BorderLayout;  
23  
24 public class BubbleSortGUI_2511533007 extends JFrame {  
25  
26     private static final long serialVersionUID = 1L;  
27     private JPanel contentPane;  
28     private int[] array_3007;  
29     private JLabel[] labelArray_3007;  
30     private JButton stepButton_3007, resetButton_3007, setButton_3007;  
31     private JTextField inputField_3007;  
32     private JPanel panelArray_3007;  
33     private JTextArea stepArea_3007;  
34  
35     private int i_3007 = 1, j_3007;  
36     private boolean sorting_3007 = false;  
37     private int stepCount_3007 = 1;  
38  
39     public BubbleSortGUI_2511533007() {  
40         setTitle("Insertion Sort Langkah per Langkah");  
41         setSize(750, 400);  
42         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
43         setLocationRelativeTo(null);  
44         setLayout(new BorderLayout());  
45  
46         // Panel input  
47         JPanel inputPanel_3007 = new JPanel(new FlowLayout());  
48         inputField_3007 = new JTextField(30);  
49         setButton_3007 = new JButton("Set Array");  
50         inputPanel_3007.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));  
51         inputPanel_3007.add(inputField_3007);  
52         inputPanel_3007.add(setButton_3007);  
53  
54         // Panel array visual  
55         JPanel panelArray_3007 = new JPanel();  
56         panelArray_3007.setLayout(new FlowLayout());  
57  
58         // Panel kontrol  
59         JPanel controlPanel_3007 = new JPanel();  
60         stepButton_3007 = new JButton("Langkah Selanjutnya");  
61         resetButton_3007 = new JButton("Reset");
```

```

62     stepButton_3007.setEnabled(false);
63     controlPanel_3007.add(stepButton_3007);
64     controlPanel_3007.add(resetButton_3007);
65
66     // Area teks untuk log langkah-langkah
67     stepArea_3007 = new JTextArea(8, 60);
68     stepArea_3007.setEditable(false);
69     stepArea_3007.setFont(new Font("Monospaced", Font.PLAIN, 14));
70     JScrollPane scrollPane_3007 = new JScrollPane(stepArea_3007);
71
72     // Tambahkan panel ke frame
73     add(inputPanel_3007, BorderLayout.NORTH);
74     add(panelArray_3007, BorderLayout.CENTER);
75     add(controlPanel_3007, BorderLayout.SOUTH);
76     add(scrollPane_3007, BorderLayout.EAST);
77
78     // Event Set Array
79     setButton_3007.addActionListener(e -> setArrayFromInput());
80
81     // Event Langkah Selanjutnya
82     stepButton_3007.addActionListener(e -> performStep_3007());
83
84     // Event Reset
85     resetButton_3007.addActionListener(e -> reset_3007());
86 }
87
88 private void setArrayFromInput() {
89     String text_3007 = inputField_3007.getText().trim();
90     if (text_3007.isEmpty()) return;
91     String [] parts_3007 = text_3007.split(",");
92     array_3007 = new int[parts_3007.length];
93     try {
94         for (int k_3007 = 0; k_3007 < parts_3007.length; k_3007++) {
95             array_3007[k_3007] = Integer.parseInt(parts_3007[k_3007].trim());
96         }
97     } catch (NumberFormatException e) {
98         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
99             + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
100         return;
101     }
102     i_3007 = 0;

```

```

103     j_3007 = 0;
104     stepCount_3007 = 1;
105     sorting_3007 = true;
106     stepButton_3007.setEnabled(true);
107     stepArea_3007.setText("");
108     panelArray_3007.removeAll();
109     labelArray_3007 = new JLabel[array_3007.length];
110     for (int k_3007 = 0; k_3007 < array_3007.length; k_3007++) {
111         labelArray_3007[k_3007] = new JLabel(String.valueOf(array_3007[k_3007]));
112         labelArray_3007[k_3007].setFont(new Font("Arial", Font.BOLD, 24));
113         labelArray_3007[k_3007].setBorder(BorderFactory.createLineBorder(Color.BLACK));
114         labelArray_3007[k_3007].setPreferredSize(new Dimension(50, 50));
115         labelArray_3007[k_3007].setHorizontalAlignment(SwingConstants.CENTER);
116         panelArray_3007.add(labelArray_3007[k_3007]);
117     }
118     panelArray_3007.revalidate();
119     panelArray_3007.repaint();
120 }
121
122 private void performStep_3007() {
123     if (!sorting_3007 || i_3007 >= array_3007.length - 1) {
124         sorting_3007 = false;
125         stepButton_3007.setEnabled(false);
126         JOptionPane.showMessageDialog(this, "Sorting selesai!");
127         return;
128     }
129     resetHighlights_3007();
130     StringBuilder stepLog = new StringBuilder();
131     labelArray_3007[j_3007].setBackground(Color.CYAN);
132     labelArray_3007[j_3007 + 1].setBackground(Color.CYAN);
133     if (array_3007[j_3007] > array_3007[j_3007 + 1]) {
134         // Swap
135         int temp = array_3007[j_3007];
136         array_3007[j_3007] = array_3007[j_3007 + 1];
137         array_3007[j_3007 + 1] = temp;
138         labelArray_3007[j_3007].setBackground(Color.RED);
139         labelArray_3007[j_3007 + 1].setBackground(Color.RED);
140         stepLog.append("Langkah ").append(stepCount_3007).append(": Menukar elemen ke-")
141             .append(j_3007).append(" (").append(array_3007[j_3007 + 1]).append(") dengan ke-")
142             .append(j_3007 + 1).append(" (").append(array_3007[j_3007]).append(")\n");
143     } else {

```

```

144         stepLog.append("Langkah ").append(stepCount_3007).append(": Tidak ada pertukaran antara ke-")
145             .append(j_3007).append(" dan ke-").append(j_3007 + 1).append("\n");
146     }
147     stepLog.append("Hasil: ").append(arrayToString_3007(array_3007)).append("\n\n");
148     stepArea_3007.append(stepLog.toString());
149     updateLabels_3007();
150     j_3007++;
151     if (j_3007 >= array_3007.length - i_3007 - 1) {
152         j_3007 = 0;
153         i_3007++;
154     }
155     stepCount_3007++;
156     if (i_3007 >= array_3007.length - 1) {
157         sorting_3007 = false;
158         stepButton_3007.setEnabled(false);
159         JOptionPane.showMessageDialog(this, "Sorting selesai!");
160     }
161 }
162
163 private void updateLabels_3007() {
164     for (int k_3007 = 0; k_3007 < array_3007.length; k_3007++) {
165         labelArray_3007[k_3007].setText(String.valueOf(array_3007[k_3007]));
166     }
167 }
168
169 private void resetHighlights_3007() {
170     for (JLabel label_3007 : labelArray_3007) {
171         label_3007.setBackground(Color.WHITE);
172     }
173 }
174
175 private void reset_3007() {
176     inputField_3007.setText("");
177     panelArray_3007.removeAll();
178     panelArray_3007.revalidate();
179     panelArray_3007.repaint();
180     stepArea_3007.setText("");
181     stepButton_3007.setEnabled(false);
182     sorting_3007 = false;
183     i_3007 = 0;
184     j_3007 = 0;
185     stepCount_3007 = 1;
186 }
187
188 private String arrayToString_3007(int[] arr) {
189     StringBuilder sb_3007 = new StringBuilder();
190     for (int k_3007 = 0; k_3007 < arr.length; k_3007++) {
191         sb_3007.append(arr[k_3007]);
192         if (k_3007 < arr.length - 1) sb_3007.append(", ");
193     }
194     return sb_3007.toString();
195 }
196
197 public static void main(String[] args) {
198     SwingUtilities.invokeLater(() -> {
199         BubbleSortGUI_2511533007 gui_3007 = new BubbleSortGUI_2511533007();
200         gui_3007.setVisible(true);
201     });
202 }
203 }

```

Kode Program 2.4

Class BubbleSortGUI_2511533007 menyediakan visualisasi untuk algoritma Bubble Sort menggunakan GUI berbasis library Java Swing. Konstruktor class ini berfungsi

membangun struktur window aplikasi, mengatur tata letak komponen visual, serta menangani fungsi penanganan klik pada setiap tombol.

Method `setArrayFromInput` bertugas mengambil input string angka dari pengguna, memisahkannya berdasarkan koma, mengubahnya menjadi array integer, serta menggambar komponen kotak label di layar.

Method `performStep_3007` mengeksekusi satu langkah perbandingan bersebelahan dari Bubble Sort setiap kali tombol ditekan, melakukan pertukaran nilai di latar belakang jika angka kiri lebih besar dari angka kanan, memberi warna highlight visual (Cyan/Red), dan mencatat aktivitasnya ke area teks log.

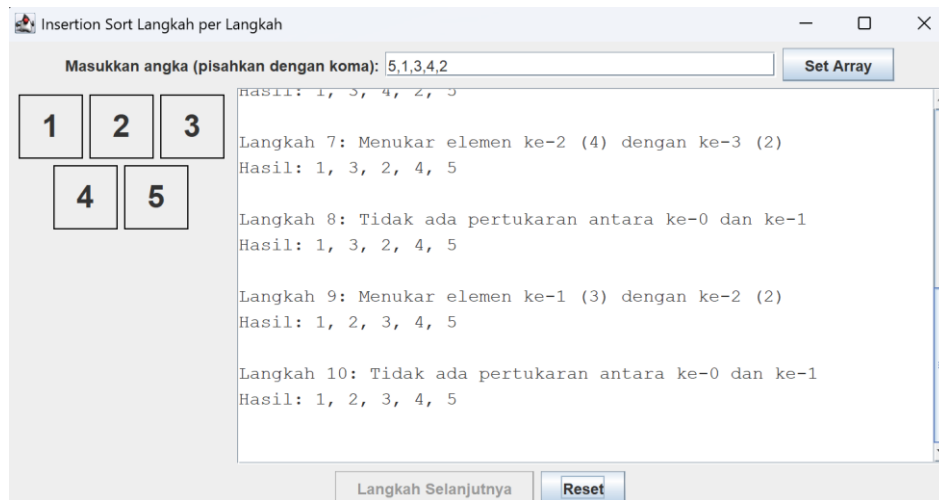
Method `updateLabels_3007` berfungsi memperbarui teks di dalam kotak label visual pada aplikasi agar perubahan posisi angka langsung terlihat oleh pengguna.

Method `resetHighlights_3007` bertugas mengembalikan warna latar belakang seluruh kotak label angka menjadi putih bersih sebelum langkah visualisasi berikutnya dimulai.

Method `reset_3007` digunakan untuk membersihkan kolom input, menghapus visualisasi kotak angka, mengosongkan riwayat log, serta mengembalikan semua status variabel kontrol ke kondisi awal.

Method `arrayToString_3007` merupakan fungsi pembantu untuk mengubah susunan data angka di dalam array menjadi sebaris teks string yang dipisahkan koma agar mudah dibaca pada log.

Terakhir, method `main` berfungsi menjalankan dan menampilkan jendela aplikasi GUI ini secara aman di dalam thread khusus penanganan event Java Swing.



2.1.5 MergeSortGUI

```

1  package pekan8_2511533007;
2
3  import java.awt.BorderLayout;
23
24  public class MergeSortGUI_2511533007 extends JFrame {
25
26      private static final long serialVersionUID = 1L;
27      private JPanel contentPane;
28      private int[] array_3007;
29      private JLabel[] labelArray_3007;
30      private JButton stepButton_3007, resetButton_3007, setButton_3007;
31      private JTextField inputField_3007;
32      private JPanel panelArray_3007;
33      private JTextArea stepArea_3007;
34
35      private int i_3007, j_3007, k_3007;
36      private boolean sorting_3007 = false;
37      private int stepCount_3007 = 1;
38      private java.util.Queue<int[]> mergeQueue_3007 = new java.util.LinkedList<>();
39      private boolean isMerging_3007 = false;
40      private int left_3007, mid_3007, right_3007;
41      private int[] temp_3007;
42      private boolean copying_3007 = false;
43
44      public MergeSortGUI_2511533007() {
45          setTitle("Insertion Sort Langkah per Langkah");
46          setSize(750, 400);
47          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
48          setLocationRelativeTo(null);
49          setLayout(new BorderLayout());
50
51          // Panel input
52          JPanel inputPanel_3007 = new JPanel(new FlowLayout());
53          inputField_3007 = new JTextField(30);
54          setButton_3007 = new JButton("Set Array");
55          inputPanel_3007.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
56          inputPanel_3007.add(inputField_3007);
57          inputPanel_3007.add(setButton_3007);
58
59          // Panel array visual
60          panelArray_3007 = new JPanel();
61          panelArray_3007.setLayout(new FlowLayout());

```

```

62
63     // Panel kontrol
64     JPanel controlPanel_3007 = new JPanel();
65     stepButton_3007 = new JButton("Langkah Selanjutnya");
66     resetButton_3007 = new JButton("Reset");
67     stepButton_3007.setEnabled(false);
68     controlPanel_3007.add(stepButton_3007);
69     controlPanel_3007.add(resetButton_3007);
70
71     // Area teks untuk log langkah-langkah
72     stepArea_3007 = new JTextArea(8, 60);
73     stepArea_3007.setEditable(false);
74     stepArea_3007.setFont(new Font("Monospaced", Font.PLAIN, 14));
75     JScrollPane scrollPane_3007 = new JScrollPane(stepArea_3007);
76
77     // Tambahkan panel ke frame
78     add(inputPanel_3007, BorderLayout.NORTH);
79     add(panelArray_3007, BorderLayout.CENTER);
80     add(controlPanel_3007, BorderLayout.SOUTH);
81     add(scrollPane_3007, BorderLayout.EAST);
82
83     // Event Set Array
84     setButton_3007.addActionListener(e -> setArrayFromInput());
85
86     // Event Langkah Selanjutnya
87     stepButton_3007.addActionListener(e -> performStep_3007());
88
89     // Event Reset
90     resetButton_3007.addActionListener(e -> reset_3007());
91 }
92
93 private void setArrayFromInput() {
94     String text_3007 = inputField_3007.getText().trim();
95     if (text_3007.isEmpty()) return;
96     String [] parts_3007 = text_3007.split(",");
97     array_3007 = new int[parts_3007.length];
98     try {
99         for (int k_3007 = 0; k_3007 < parts_3007.length; k_3007++) {
100             array_3007[k_3007] = Integer.parseInt(parts_3007[k_3007].trim());
101         }
102     } catch (NumberFormatException e) {

```

```

103         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
104             + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
105         return;
106     }
107     labelArray_3007 = new JLabel[array_3007.length];
108     panelArray_3007.removeAll();
109     for (int i_3007 = 0; i_3007 < array_3007.length; i_3007++) {
110         labelArray_3007[i_3007] = new JLabel(String.valueOf(array_3007[i_3007]));
111         labelArray_3007[i_3007].setFont(new Font("Arial", Font.BOLD, 24));
112         labelArray_3007[i_3007].setOpaque(true);
113         labelArray_3007[i_3007].setBackground(Color.WHITE);
114         labelArray_3007[i_3007].setBorder(BorderFactory.createLineBorder(Color.BLACK));
115         labelArray_3007[i_3007].setPreferredSize(new Dimension(50, 50));
116         labelArray_3007[i_3007].setHorizontalAlignment(SwingConstants.CENTER);
117         panelArray_3007.add(labelArray_3007[i_3007]);
118     }
119     mergeQueue_3007.clear();
120     generateMergeSteps_3007(0, array_3007.length - 1);
121     stepButton_3007.setEnabled(true);
122     stepArea_3007.setText("");
123     stepCount_3007 = 1;
124     isMerging_3007 = false;
125     panelArray_3007.revalidate();
126     panelArray_3007.repaint();
127 }
128 private void generateMergeSteps_3007(int left_3007, int right_3007) {
129     if (left_3007 < right_3007) {
130         int mid_3007 = left_3007 + (right_3007 - left_3007) / 2;
131
132         generateMergeSteps_3007(left_3007, mid_3007);
133         generateMergeSteps_3007(mid_3007 + 1, right_3007);
134
135         mergeQueue_3007.add(new int[] { left_3007, mid_3007, right_3007 });
136     }
137 }
138
139 private void performStep_3007() {
140     resetHighlights_3007();
141
142     if (!isMerging_3007 && !mergeQueue_3007.isEmpty()) {
143         int [] range_3007 = mergeQueue_3007.poll();
144         left_3007 = range_3007[0];

```



```

145         mid_3007 = range_3007[1];
146         right_3007 = range_3007[2];
147         temp_3007 = new int[right_3007 - left_3007 + 1];
148         i_3007 = left_3007;
149         j_3007 = mid_3007 + 1;
150         k_3007 = 0;
151         copying_3007 = false;
152         isMerging_3007 = true;
153         stepArea_3007.append("Langkah " + stepCount_3007++ + ": Mulai merge dari " + left_3007 + " ke " + right_3007 + "\n");
154         return;
155     }
156
157     if (isMerging_3007 && !copying_3007) {
158         if (i_3007 <= mid_3007 && j_3007 <= right_3007) {
159             labelArray_3007[i_3007].setBackground(Color.CYAN);
160             labelArray_3007[j_3007].setBackground(Color.CYAN);
161             if (array_3007[i_3007] <= array_3007[j_3007]) {
162                 temp_3007[k_3007++] = array_3007[i_3007++];
163             } else {
164                 temp_3007[k_3007++] = array_3007[j_3007++];
165             }
166             stepArea_3007.append("Langkah " + stepCount_3007++ + ": Bandingkan dan saling elemen\n");
167             return;
168         } else if (i_3007 <= mid_3007) {
169             temp_3007[k_3007++] = array_3007[i_3007++];
170             stepArea_3007.append("Langkah " + stepCount_3007++ + ": Salin sisa kanan\n");
171             return;
172         } else if (j_3007 <= right_3007) {
173             temp_3007[k_3007++] = array_3007[j_3007++];
174             stepArea_3007.append("Langkah " + stepCount_3007++ + ": Salin sisa kanan\n");
175             return;
176         } else {
177             copying_3007 = true;
178             k_3007 = 0;
179             return;
180         }
181     }
182
183     if (copying_3007 && k_3007 < temp_3007.length) {
184         array_3007[left_3007 + k_3007] = temp_3007[k_3007];
185         labelArray_3007[left_3007 + k_3007].setText(String.valueOf(temp_3007[k_3007]));
186         labelArray_3007[left_3007 + k_3007].setBackground(Color.GREEN);

```

```

187         k_3007++;
188         stepArea_3007.append("Langkah " + stepCount_3007++ + ": Tempelkan ke array utama\n");
189         return;
190     }
191
192     if (copying_3007 && k_3007 == temp_3007.length) {
193         isMerging_3007 = false;
194         copying_3007 = false;
195     }
196
197     if (mergeQueue_3007.isEmpty() && !isMerging_3007) {
198         stepArea_3007.append("Selesai.\n");
199         stepButton_3007.setEnabled(false);
200         JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
201     }
202 }
203
204 private void resetHighlights_3007() {
205     if (labelArray_3007 == null) return;
206     for (JLabel label_3007 : labelArray_3007) {
207         label_3007.setBackground(Color.WHITE);
208     }
209 }
210
211 private void reset_3007() {
212     inputField_3007.setText("");
213     panelArray_3007.removeAll();
214     panelArray_3007.revalidate();
215     panelArray_3007.repaint();
216     stepArea_3007.setText("");
217     stepButton_3007.setEnabled(false);
218     mergeQueue_3007.clear();
219     isMerging_3007 = false;
220     stepCount_3007 = 1;
221 }
222
223
224 private String arrayToString_3007(int[] arr_3007) {
225     StringBuilder sb_3007 = new StringBuilder();
226     for (int k_3007 = 0; k_3007 < arr_3007.length; k_3007++) {
227         sb_3007.append(arr_3007[k_3007]);
228         if (k_3007 < arr_3007.length - 1) sb_3007.append(", ");
229     }
230     return sb_3007.toString();
231 }
232
233 public static void main(String[] args) {
234     SwingUtilities.invokeLater(() -> {
235         MergeSortGUI_2511533007 gui_3007 = new MergeSortGUI_2511533007();
236         gui_3007.setVisible(true);
237     });
238 }
239 }

```

Kode Program 2. 5

Class MergeSortGUI_2511533007 menyediakan visualisasi untuk algoritma Merge Sort menggunakan GUI berbasis library Java Swing. Konstruktor class ini berfungsi membangun struktur window aplikasi, mengatur tata letak komponen visual, serta menangani fungsi penangan klik pada setiap tombol.

Method `setArrayFromInput` bertugas mengambil input string angka dari pengguna, memisahkannya berdasarkan koma, mengubahnya menjadi array integer, menyiapkan struktur antrean data, serta menggambar komponen kotak label di layar.

Method `generateMergeSteps_3007` berfungsi memetakan seluruh urutan indeks pembagian subarray kiri dan kanan secara rekursif di awal, lalu memasukkan koordinat batas penggabungan tersebut ke dalam struktur antrean (Queue).

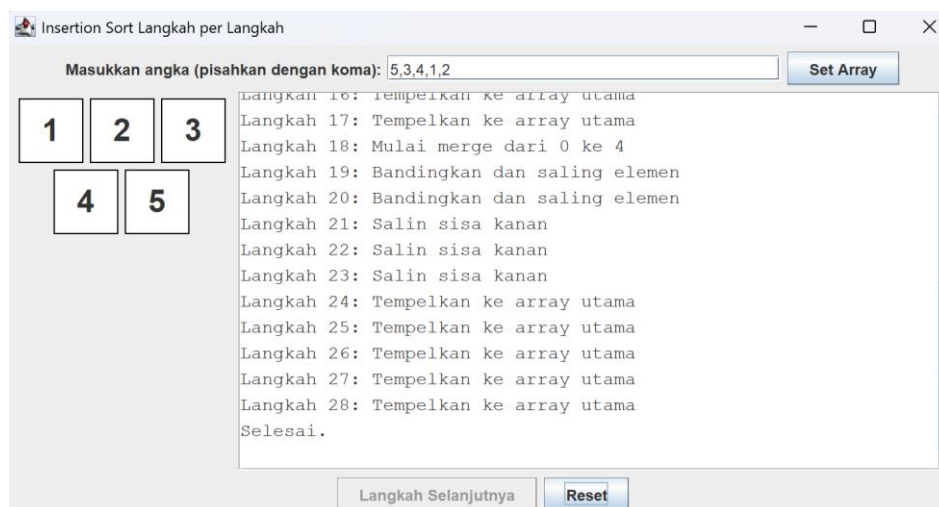
Method `performStep_3007` mengeksekusi satu bagian terkecil dari proses Merge Sort setiap kali tombol ditekan, mulai dari mengaktifkan area penggabungan baru, membandingkan elemen antar subarray dengan penanda warna Cyan, menyalin sisa elemen, hingga memindahkan data dari array temporer ke array utama dengan highlight warna Hijau sembari mencatatnya ke teks log.

Method `resetHighlights_3007` bertugas mengembalikan warna latar belakang seluruh kotak label angka menjadi putih bersih sebelum langkah visualisasi berikutnya dimulai.

Method `reset_3007` digunakan untuk membersihkan kolom input, menghapus visualisasi kotak angka, mengosongkan antrean pemisahan, serta mengembalikan semua status variabel kontrol ke kondisi awal.

Method `arrayToString_3007` merupakan fungsi pembantu untuk mengubah susunan data angka di dalam array menjadi sebaris teks string yang dipisahkan koma agar mudah dibaca pada log.

Terakhir, method `main` berfungsi menjalankan dan menampilkan jendela aplikasi GUI ini secara aman di dalam thread khusus penanganan event Java Swing.



BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa algoritma pengurutan data memiliki karakteristik, mekanisme kerja, dan tingkat efisiensi yang berbeda-beda dalam menyelesaikan masalah restrukturisasi data. Algoritma Merge Sort dan Quick Sort berbasis pendekatan divide and conquer terbukti sangat efisien untuk menangani kumpulan data skala besar berkat kemampuannya membagi masalah menjadi sub-masalah yang lebih kecil, di mana penggunaan strategi Median of Three pada Quick Sort mampu mengoptimalkan penentuan pivot demi menghindari kompleksitas terburuk. Di sisi lain, Shell Sort hadir sebagai bentuk optimalisasi dari Insertion Sort yang memanfaatkan mekanisme jarak lompatan (gap) untuk mempercepat pemindahan elemen yang letaknya berjauhan, sehingga mampu mereduksi waktu komparasi secara signifikan dibandingkan metode konvensional lainnya.

Selain pemahaman algoritma secara struktural berbasis console, praktikum ini juga berhasil mengimplementasikan visualisasi interaktif melalui pembuatan aplikasi berbasis Graphical User Interface (GUI) menggunakan library Java Swing untuk algoritma Bubble Sort dan Merge Sort. Melalui representasi visual interaktif step by step yang dilengkapi dengan fitur highlighting warna serta pencatatan log aktivitas, proses abstrak seperti pertukaran elemen (swapping) pada Bubble Sort maupun pemisahan dan penggabungan kembali subarray pada Merge Sort menjadi jauh lebih mudah dipahami secara intuitif. Dengan demikian, pembuatan program berbasis GUI ini tidak hanya memperkuat pemahaman logika pengurutan secara teoritis, tetapi juga memberikan pengalaman praktis dalam merancang antarmuka aplikasi simulasi edukasi yang responsif di dalam thread Java Swing.

DAFTAR PUSTAKA

- [1] Telkom University Jakarta, "Sorting Lanjutan: Merge Sort dan Quick Sort," 1 Desember 2023. [Daring]. Tersedia pada: <https://jakarta.telkomuniversity.ac.id/sorting-lanjutan-merge-sort-dan-quick-sort/>. [Diakses: 05-Juni-2026].
- [2] BINUS University Bandung, "Algoritma Quick Sort di Python," 23 Desember 2019. [Daring]. Tersedia pada: <https://binus.ac.id/bandung/2019/12/algoritma-quick-sort-di-python/>. [Diakses: 05-Juni-2026].
- [3] GeeksforGeeks, "Shell Sort - Data Structures and Algorithms Tutorials," 22 April 2024. [Daring]. Tersedia pada: <https://www.geeksforgeeks.org/dsa/shell-sort/>. [Diakses: 05-Juni-2026].

BAB IV Laporan Tugas

A. Kode ADT

```
1 package pekan8_2511533007;
2
3 public class Lagu_2511533007 {
4     private String judul_3007;
5     private String penyanyi_3007;
6     private int durasi_3007;
7
8     // Konstruktor
9     public Lagu_2511533007(String judul_3007, String penyanyi_3007, int durasi_3007) {
10         this.judul_3007 = judul_3007;
11         this.penyanyi_3007 = penyanyi_3007;
12         this.durasi_3007 = durasi_3007;
13     }
14
15     // Getter
16     public String getJudul_3007() { return judul_3007; }
17     public String getPenyanyi_3007() { return penyanyi_3007; }
18     public int getDurasi_3007() { return durasi_3007; }
19
20     // Setter
21     public void setJudul_3007(String judul_3007) { this.judul_3007 = judul_3007; }
22     public void setPenyanyi_3007(String penyanyi_3007) { this.penyanyi_3007 = penyanyi_3007; }
23     public void setDurasi_3007(int durasi_3007) { this.durasi_3007 = durasi_3007; }
24
25     public String toString_3007() {
26         return judul_3007 + " - " + durasi_3007 + " detik";
27     }
28 }
```

B. Kode Program

```

1 package pekan8_2511533007;
2
3 import java.util.Scanner;
4
5 public class Sorting_2511533007 {
6     // Array dataLagu untuk menyimpan maksimal 20 lagu
7     Lagu_2511533007[] dataLagu_3007 = new Lagu_2511533007[20];
8     int jumlah_3007 = 0;
9
10    public void inputData_3007(Scanner scanner_3007) {
11        // Validasi penentuan jumlah lagu sesuai batas maksimal tampungan array (20)
12        while (true) {
13            try {
14                System.out.print("Masukkan jumlah lagu yang ingin diinput (min 7, maks 20): ");
15                jumlah_3007 = Integer.parseInt(scanner_3007.nextLine());
16
17                if (jumlah_3007 >= 7 && jumlah_3007 <= 20) {
18                    break;
19                } else {
20                    System.out.println("Error, Jumlah lagu harus di antara 7 dan 20");
21                }
22            } catch (NumberFormatException e) {
23                System.out.println("Error, Input tidak valid, Harap masukkan angka bulat");
24            }
25        }
26
27        System.out.println("\n--- Silakan Masukkan Data Lagu ---");
28        for (int i_3007 = 0; i_3007 < jumlah_3007; i_3007++) {
29            System.out.println("Lagu ke" + (i_3007 + 1) + ":");
30
31            // Input dan validasi Judul
32            String judul_3007 = "";
33            while (true) {
34                System.out.print("    Judul      : ");
35                judul_3007 = scanner_3007.nextLine().trim();
36                if (!judul_3007.isEmpty()) {
37                    break;
38                }
39                System.out.println("    Error, Judul lagu tidak boleh kosong");
40            }
41
42            // Input dan validasi Penyanyi

```

```

43     String penyanyi_3007 = "";
44     while (true) {
45         System.out.print("  Penyanyi: ");
46         penyanyi_3007 = scanner_3007.nextLine().trim();
47         if (!penyanyi_3007.isEmpty()) {
48             break;
49         }
50         System.out.println("  Error, Nama penyanyi tidak boleh kosong");
51     }
52
53     // Input dan validasi Durasi
54     int durasi_3007 = 0;
55     while (true) {
56         try {
57             System.out.print("  Durasi: ");
58             durasi_3007 = Integer.parseInt(scanner_3007.nextLine());
59             if (durasi_3007 > 0) {
60                 break;
61             }
62             System.out.println("  Error,Durasi harus lebih besar dari 0 detik");
63         } catch (NumberFormatException e) {
64             System.out.println("  Error, Input tidak valid, Harap masukkan angka bulat");
65         }
66     }
67
68     // Membuat objek lagu baru berdasarkan input user ke dalam array
69     dataLagu_3007[i_3007] = new Lagu_2511533007(judul_3007, penyanyi_3007, durasi_3007);
70 }
71
72
73 // Method untuk menampilkan data sebelum dan sesudah sorting
74 public void tampilData_3007() {
75     for (int i_3007 = 0; i_3007 < jumlah_3007; ++i_3007) {
76         System.out.println((i_3007 + 1) + ". " + dataLagu_3007[i_3007].toString_3007());
77     }
78 }
79
80 // Swap dua elemen array lagu
81 static void swap_3007(Lagu_2511533007[] arr_3007, int i_3007, int j_3007) {
82     Lagu_2511533007 temp_3007 = arr_3007[i_3007];
83     arr_3007[i_3007] = arr_3007[j_3007];

```



```

84     arr_3007[j_3007] = temp_3007;
85 }
86
87 // Metode tambahan untuk mengatur pivot menggunakan median-of-three
88 static void medianOfThree_3007(Lagu_2511533007[] arr_3007, int low_3007, int high_3007) {
89     int mid_3007 = low_3007 + (high_3007 - low_3007) / 2;
90
91     // Urutkan berdasarkan durasi: low, mid, high
92     if (arr_3007[low_3007].getDurasi_3007() > arr_3007[mid_3007].getDurasi_3007()) {
93         swap_3007(arr_3007, low_3007, mid_3007);
94     }
95     if (arr_3007[low_3007].getDurasi_3007() > arr_3007[high_3007].getDurasi_3007()) {
96         swap_3007(arr_3007, low_3007, high_3007);
97     }
98     if (arr_3007[mid_3007].getDurasi_3007() > arr_3007[high_3007].getDurasi_3007()) {
99         swap_3007(arr_3007, mid_3007, high_3007);
100    }
101    // Pindahkan median ke posisi high sebagai pivot
102    swap_3007(arr_3007, mid_3007, high_3007);
103 }
104
105 // Method partisi mendasarkan pada durasi lagu
106 static int partition_3007(Lagu_2511533007[] arr_3007, int low_3007, int high_3007) {
107     medianOfThree_3007(arr_3007, low_3007, high_3007);
108
109     int pivot_3007 = arr_3007[high_3007].getDurasi_3007();
110     int i_3007 = (low_3007 - 1);
111
112     for (int j_3007 = low_3007; j_3007 <= high_3007 - 1; j_3007++) {
113         if (arr_3007[j_3007].getDurasi_3007() < pivot_3007) {
114             i_3007++;
115             swap_3007(arr_3007, i_3007, j_3007);
116         }
117     }
118     swap_3007(arr_3007, i_3007 + 1, high_3007);
119     return (i_3007 + 1);
120 }
121
122 // Method utama Quick Sort secara rekursif
123 static void quickSort_3007(Lagu_2511533007[] arr_3007, int low_3007, int high_3007) {
124     if (low_3007 < high_3007) {

```

```

125         int pi_3007 = partition_3007(arr_3007, low_3007, high_3007);
126         quickSort_3007(arr_3007, low_3007, pi_3007 - 1);
127         quickSort_3007(arr_3007, pi_3007 + 1, high_3007);
128     }
129 }
130
131 public void quickSort_3007() {
132     if (jumlah_3007 > 1) {
133         quickSort_3007(dataLagu_3007, 0, jumlah_3007 - 1);
134     }
135 }
136
137 public static void main(String[] args) {
138     Scanner scanner_3007 = new Scanner(System.in);
139     Sorting_2511533007 playlist_3007 = new Sorting_2511533007();
140
141     System.out.println("=== Sorting Playlist NIM: 2511533007 ===");
142     playlist_3007.inputData_3007(scanner_3007);
143
144     System.out.println("\nData Sebelum Sorting:");
145     playlist_3007.tampilData_3007();
146
147     playlist_3007.quickSort_3007();
148
149     System.out.println("\nData Setelah Quick Sort:");
150     playlist_3007.tampilData_3007();
151
152     scanner_3007.close();
153 }
154 }

```

C. Alasan memilih

Pemilihan algoritma Quick Sort didasarkan pada efisiensi kinerjanya yang sangat tinggi dalam mengurutkan data bertipe array di memori lokal. Algoritma ini membagi urutan masalah playlist menggunakan strategi divide and conquer sehingga proses komparasi durasi lagu menjadi lebih terfokus pada submasalah yang kecil. Selain itu, implementasi di atas telah dilengkapi dengan optimasi teknik Median of Three yang memilih nilai tengah dari elemen awal, tengah, dan akhir sebagai pivot. Modifikasi tersebut sukses meminimalkan peluang terjadinya pembagian data yang timpang, sehingga program terhindar dari skenario terburuk dan performa pengurutan playlist tetap berjalan stabil dan optimal.

D. Kompleksitas Algoritma

Secara teoritis, Quick Sort dengan optimasi Median of Three memiliki kompleksitas waktu rata-rata (average case) dan kondisi terbaik (best case) sebesar $O(n \log n)$, yang membuatnya jauh lebih cepat dibandingkan algoritma konvensional berderajat kuadratik. Pada kondisi terburuk, kompleksitasnya bisa mencapai $O(n^2)$, namun probabilitas ini telah direduksi seminimal mungkin berkat penentuan pivot berbasis median. Sementara itu, kompleksitas ruang algoritma ini adalah $O(\log n)$ karena memanfaatkan tumpukan rekursif memori tanpa memerlukan alokasi array baru tambahan di luar struktur aslinya.

E. Penjelasan Kode

Konstruktor `Lagu_2511533007` berfungsi sebagai cetakan utama untuk menginisialisasi objek lagu baru dengan cara memetakan argumen judul, penyanyi, dan durasi ke dalam atribut privat kelas. Untuk menjembatani akses data karena adanya pembatasan hak akses (encapsulation), disediakan sekelompok method Getter seperti `getJudul_3007`, `getPenyanyi_3007`, dan `getDurasi_3007` yang bertugas mengambil nilai atribut dari luar kelas, di mana `getDurasi_3007` memegang peran krusial sebagai indikator pembandingan nilai dalam proses pengurutan. Sebaliknya, kelompok method Setter bertugas untuk mengubah atau memperbarui data atribut objek secara dinamis jika diperlukan perubahan di latar belakang. Sementara itu, method `toString_3007` bertindak sebagai pengatur format tampilan objek, yang menyatukan informasi judul dan durasi ke dalam sebaris teks string terstruktur agar siap dicetak dengan rapi pada jendela keluaran (output).

Method `inputData_3007` berfungsi untuk mengalokasikan data sampel awal berupa 7 objek lagu acak ke dalam array `dataLagu_3007` sekaligus mencatat jumlah total lagu yang aktif ke dalam variabel `jumlah_3007`. Untuk menyajikan data tersebut ke pengguna, method `tampilData_3007` bekerja melakukan perulangan dari indeks awal hingga batas jumlah lagu untuk mencetak daftar playlist secara berurutan lengkap dengan nomor urutnya. Dalam proses manipulasi posisi data, method utilitas `swap_3007` memproses pertukaran tempat antara dua objek lagu di dalam array menggunakan bantuan satu variabel penampung sementara. Langkah pengurutan kemudian dioptimalkan oleh method `medianOfThree_3007` yang bertugas memeriksa

elemen di posisi awal, tengah, dan akhir subarray untuk mencari nilai median durasi, lalu memindahkannya ke batas kanan sebagai elemen pembatas (pivot).

Selanjutnya, penataan elemen diatur oleh method `partition_3007` yang memisahkan barisan data berdasarkan pivot durasi lagu tersebut; elemen yang lebih pendek dari pivot dikelompokkan ke sisi kiri dan yang lebih panjang ke sisi kanan, lalu method ini akan mengembalikan posisi indeks pivot yang baru. Fungsi inti pengurutan dijalankan oleh method rekursif `quickSort_3007` dengan parameter, yang bekerja membelah sub-masalah playlist secara terus-menerus ke area kiri dan kanan berdasarkan hasil partisi hingga seluruh data sukses terurut. Agar pemanggilan fungsi ini lebih praktis dari fungsi utama, disediakan wrapper method `quickSort_3007` tanpa parameter yang bertugas mengotomatisasi perhitungan batas indeks awal dan akhir playlist. Terakhir, method `main` bertindak sebagai pengendali utama yang bertugas menginisialisasi objek playlist, mengeksekusi pengisian data, serta menampilkan visualisasi perbandingan playlist sebelum dan sesudah diurutkan berdasarkan durasi terkecil.

Output Kode

```
=== Sorting Playlist NIM: 2511533007 ===
Masukkan jumlah lagu yang ingin diinput (min 7, maks 20): 14

--- Silakan Masukkan Data Lagu ---
Lagu ke1:
  Judul      : qwert
  Penyanyi: py
  Durasi: 123
Lagu ke2:
  Judul      : qwerty
  Penyanyi: poi
  Durasi: 124
Lagu ke3:
  Judul      : wertyyu
  Penyanyi: iuyih
  Durasi: 125
Lagu ke4:
  Judul      : oxndu
  Penyanyi: xgdkd
  Durasi: 126
Lagu ke5:
  Judul      : lcydsmc
  Penyanyi: kcfckr
  Durasi: 127
Lagu ke6:
  Judul      : pxrsams
  Penyanyi: nzrtfbr
  Durasi: 128
Lagu ke7:
  Judul      : jzxuysp
  Penyanyi: qwzvbbvi
  Durasi: 129
Lagu ke8:
  Judul      : bnzwah
  Penyanyi: pydgd
  Durasi: 120
Lagu ke9:
  Judul      : ghxt
  Penyanyi: mvvc1
  Durasi: 134
```

```

Lagu ke10:
  Judul      : sabana
  Penyanyi: judul
  Durasi: 135
Lagu ke11:
  Judul      : juduy
  Penyanyi: laguy
  Durasi: 126
Lagu ke12:
  Judul      : bwahbwah
  Penyanyi: uiuiuiu
  Durasi: 198
Lagu ke13:
  Judul      : papopoi
  Penyanyi: pulupulu
  Durasi: 145
Lagu ke14:
  Judul      : ujn
  Penyanyi: edcv
  Durasi: 178
|
Data Sebelum Sorting:
1. qwert - 123 detik
2. qwerty - 124 detik
3. wertyyu - 125 detik
4. oxndu - 126 detik
5. lcydsmc - 127 detik
6. pxrsams - 128 detik
7. jzxuysp - 129 detik
8. bnzwah - 120 detik
9. ghxt - 134 detik
10. sabana - 135 detik
11. juduy - 126 detik
12. bwahbwah - 198 detik
13. papopoi - 145 detik
14. ujn - 178 detik

Data Setelah Quick Sort:
1. bnzwah - 120 detik
2. qwert - 123 detik
3. qwerty - 124 detik
4. wertyyu - 125 detik
5. oxndu - 126 detik
6. juduy - 126 detik
7. lcydsmc - 127 detik
8. pxrsams - 128 detik
9. jzxuysp - 129 detik
10. ghxt - 134 detik
11. sabana - 135 detik
12. papopoi - 145 detik
13. ujn - 178 detik
14. bwahbwah - 198 detik

```